A Minor Project Final Report on

A Real Time Sign Language Recognition System

Submitted in Partial Fulfillment of the Requirements

for the Degree of **Computer Engineering**

under Pokhara University

Submitted by:

Aadarsha Sapkota, 211301

Saimon Thapa, 211328

Sunny Jha, 211338

Ritesh Shrestha, 211343

Under the Supervision of

Mr. Saroj Shakya

Date:

February 21, 2025



Department of Computer Engineering

NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur, Nepal

Acknowledgement

We would like to express my deepest gratitude to our team members who contributed to the successful completion of this project, Real time sign language recognition system.

First and foremost, we would like to thank our project supervisor, Mr. Saroj Shakya, for his invaluable guidance, encouragement, and constructive feedback throughout the project. His expertise and support were pivotal in helping us navigate challenges and refine our work.

We are also grateful to Nepal college of information technology and the Department of Computer Engineering for providing the necessary infrastructure and resources to carry out this project. Special thanks to our professors and mentors for their advice and insights, which helped shape the direction of our work.

We would like to extend my heartfelt thanks to my peers and classmates for their moral support and collaborative spirit. Their discussions and suggestions were instrumental in enhancing our understanding and perspective.

Abstract

Sign language serves as a crucial means of communication for individuals with hearing or speech impairments. However, the lack of widespread knowledge of sign language often creates communication barriers. This project aims to bridge this gap by developing a real-time sign language recognition system capable of recognizing and translating American Sign Language (ASL) gestures into textual representations. The proposed system focuses on the recognition of ASL letters A-C using a Convolutional Neural Network (CNN) model. The CNN is trained using TensorFlow and Keras on a dataset, including gestures for A, B, C, and a blank class to represent no gesture. The system leverages a webcam interface to capture live video frames, which are processed in real time to identify the gestures shown by the user. A Flask-based web application integrates the model, providing a user-friendly interface where the recognized gesture is displayed as text on the screen.

Keywords: Convolutional Neural Network (CNN),TensorFlow,Keras,Flask

Table of Contents

List of Figures **III**

List of Tables **IV**

1 Introduction **1**

2 Problem Statement **2**

3 Project Objectives **3**

4 Scope **4**

4.1 Project Scope 4

5 Significance of study **5**

5.1 Bridge the communication gap 5

5.2 Employment and Professional Development 5

6 Literature Review **6**

7 Methodology **7**

7.1 Software Process Model 7

7.2 Convolutional Neural Network 9

7.3 Tools and Technology 10

8 UML Diagrams **11**

8.1 UML Diagrams 11

8.1.1 System Architecture Diagram 11

8.1.2 Use Case Diagram 12

8.1.3 Flow Chart 13

8.1.4 Sequence Diagram 14

8.1.5 Activity Diagram 15

9 Task Done **16**

9.1 Collection Of Data 16

9.2 Splitting The Data 17

9.3 Creating a web interface using flask 17
9.4 Image Preprocessing 18
9.5 Creating The CNN model 18
9.6 Training The model 19
9.7 Combining Everything 19

10 PERFORMANCE ANALYSIS AND VALIDATION 20
10.1 Visualization of our model 24

11 Deliverable 25

12 Conclusion 26

13 Further works 27
13.1 Dataset Expansion 27
13.2 Improve model accuracy 27
13.3 Integration on various field 27
13.4 Real time implementation and feedback 27

14 Project Task and Time Schedule 28

References 29

Appendix 30

List of Figures

1 Iterative Model 8
2 System Architecture Diagram 11
3 Use Case Diagram 12
4 Flow chart 13
5 Sequence Diagram 14
6 Activity Diagram 15
7 Sample of Dataset 16
8 Epoch Accuracy 20
9 Epoch Loss 21
10 Evaluation Accuracy vs Iterations 22
11 Evaulation loss vs Iterations 23
12 CNN model visualization 24
13 Gantt Chart for Iteration 1 28
14 Gantt Chart for Iteration 2 28
15 Gantt Chart for Iteration 3 28
16 Recognition of A 30
17 Recognition of B 31
18 Recognition of C 32

List of Tables

1 Tools and Technologies Used 10

1 Introduction

Communication is defined as the act of sharing or exchanging information, ideas or feelings. To establish communication between two people, both of them are required to have knowledge and understanding of a common language. But in the case of deaf and dumb people, the means of communication are different. Deaf is the inability to hear and dumb is the inability to speak. They communicate using sign language among themselves and with normal people but normal people do not take seriously the importance of sign language. Not everyone possesses the knowledge and understanding of sign language which makes communication difficult between a normal person and a deaf and dumb person.

To address this need, we propose the development and launch of an model based on machine learning. The proposed system would be a real time system wherein live sign gestures would be processed using image processing. Then classifiers would be used to differentiate various signs and the translated output would be displaying text. Machine Learning algorithms will be used to train on the data set. The purpose of the system is to improve the existing system in this area in terms of response time and accuracy with the use of efficient algorithms, high quality data sets and better sensors. This document outlines the project's scope, features, technical specifications, and overall development plan.

2 Problem Statement

In context of Nepal there exists a very few system that deals with this problem. This lack of system that deals with this problem is making it hard for people to communicate in common places like school,markets,workplace and any public service place where instant communication is needed. Even the existing few systems have low accuray,lack real time performance and not practical to use for general people. In addition they also struggle with recognizing signs perfomed by different individuals with varying hand shape and gesturing styles.

This project aims to solve this existing problem by developing a new sign language detection system using OpenCV and python.Our aim is to create a system that can recognize sign language from a users video and providing with real time output.

3 Project Objectives

1. To create a comprehensive and reliable system for sign language detection.

2. To provide differently abled people streamlined platform to efficiently communicate with other individuals.

4 Scope

This section outlines the key functionalities included in the project scope.

4.1 Project Scope

Here is an overview of the scope of our sign language recognition system:

1. **Public Integration:**
    - The system will explore the possibility of integration of a sytem in public places like school,public service etc.
2. **Full sentence recognization:**
    - We will try to make system that will be able to undersatnd the whole sentence rather than just a individual letters.
3. **High Accuracy Rate:**
    - For the improvement system willmuse feedback mechanism for the further improvement over time by learning from users.

5 Significance of study

5.1 Bridge the communication gap

By providing a reliable and real-time sign language detection system, our project aims to bridge the communication gap between individuals with hearing impairments and non- sign language users.

5.2 Employment and Professional Development

The ability to effectively communicate and convey ideas is crucial for professional growth and career opportunities. Our project opens up avenues for individuals with hearing impairments to participate in various professional domains.

6 Literature Review

Various research and work has been done in this field for many years and massive improvement has been there.

The first ever approach to sign language detection was done by Bergh in 2011 . A hand gesture recognition system was built which gave good results but it only considered six gestures.

The research paper titled “Sign language Translation”[2] by authors Harini R, Janani R,Keerthana S and Madhubala S proposed system that contains four modules such as: image capturing, preprocessing classification and prediction. By using image processing the segmentation was done. Sign gestures are captured and processed using OpenCV python library. The captured gesture is resized, converted to grey scale image and the noise is filtered to achieve prediction with high accuracy.

Surejya Suresh[3] et al built a model that has classified 6 different sign languages using two different optimizers, SGD and Adam with an accuracy of 99.12

Sreyasi Dutta et al built the Sign Language Detection using Action Recognition with Python[1]. The trained LSTM model of their system demonstrated high accuracy and performance during evaluation, as evidenced by metrics such as accuracy, precision, recall, and F1-score. They were also able to implement in real time on a video stream. This implementation showcased that the model is a practical and efficient tool that can be used for communication between individuals with hearing and speaking impairement and the wider community.

7 Methodology

We have planned to work following these methodologies for the application of knowledge, skills, and technique to a wide range of activities to meet the requirements of our project

7.1 Software Process Model

. The framework used for developing this project is the iterative model of the software development life cycle. In this model, a simple and primitive implementation of a very small set of software requirements is done at first, which is followed by the iterative enhancement in the primitive model until all requirements are fulfilled and the software is ready for deployment. The following sub-section briefly describes various phases in iterative model of SDLC that was applied in the development of system.

- **First Iteration:** The objective of the first iteration will be to create a basic and functional prototype to recognize hand gestures. This phase will include tasks such as dataset preparation, model architecture design, training and evaluation and testing
- **Second Iteration :** The objective of second iteration would be to improve accuracy, usability and adaptability. This phase would include tasks such as dataset expansion, model optimization, user interface development, and performance optimization.
- **Third Iteration :** The objective of third iteration would be to finalize the system for real world application and scalability.

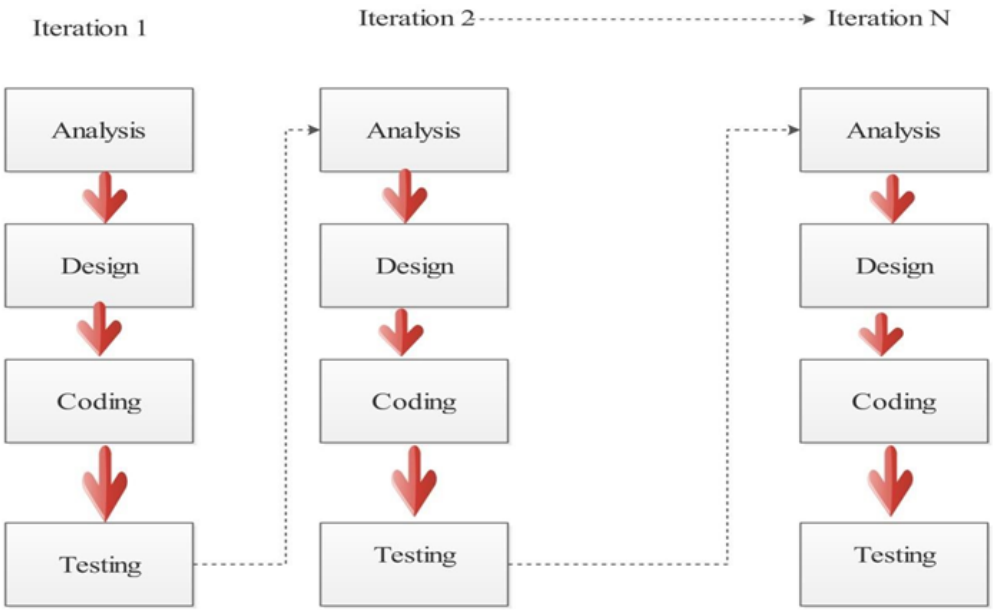


Figure 1: Iterative Model

Each phase will be developed, tested, and released separately, allowing for early feedback and continuous improvements.

7.2 Convolutional Neural Network

A convolutional neural network (CNN) is a type of artificial neural network which uses the perceptron learning rule along with supervised learning to analyze data. CNNs are applied to process images, natural language, and other kinds of cognitive tasks. Like other artificial neural networks, a convolutional neural network contains an input layer, an output layer, and various hidden layers. Some of these layers are convolutional, using a mathematical model to pass on results to successive layers. This simulates some of the actions in the human visual cortex. CNN is a fundamental example of a deep learning algorithm.

Why CNN for Gesture Recognition?

Feature Extraction: CNNs can identify patterns like edges, shapes, and textures from images, which are crucial for recognizing hand gestures.
Accuracy: CNNs achieve high accuracy in image classification tasks due to their layered structure and ability to learn complex features.
End-to-End Learning: They eliminate the need for manual feature extraction by automatically learning the most relevant features during training.

How the Algorithm Works

Input Layer: Takes the image of a hand gesture captured by the webcam.
Convolutional Layer: Extracts key features from the image using filters, highlighting the unique patterns of each gesture.
Pooling Layer: Reduces the size of the feature maps, making the model faster and more efficient while retaining important features.
Fully Connected Layer: Combines the features to make a prediction about the gesture.
Output Layer: Produces a probability distribution over all possible gestures, identifying the one with the highest likelihood.

7.3 Tools and Technology

The tools and technology used for the development of this platform are described below:

Tool/Technology	Description
Python	Used as the programming language
TensorFlow	Library for dataflow
Keras	Library for neural networks
OpenCV	For real-time computer vision
NumPy	Library for mathematical operations
Web Camera	For user input
GitHub	For version control and collaboration

Table 1: Tools and Technologies Used

8 UML Diagrams

8.1 UML Diagrams

The UML diagram of this platform is illustrated below with details.

8.1.1 System Architecture Diagram

System design is a sequence of steps that enables the designer to describe all aspects of the software to be built. We have made an attempt to make sure that the system design complies with the user requirements of the system. For this we have frequently looked upon: Verification of input and output format makes sure that the user requirements are met, and security requirements are met.

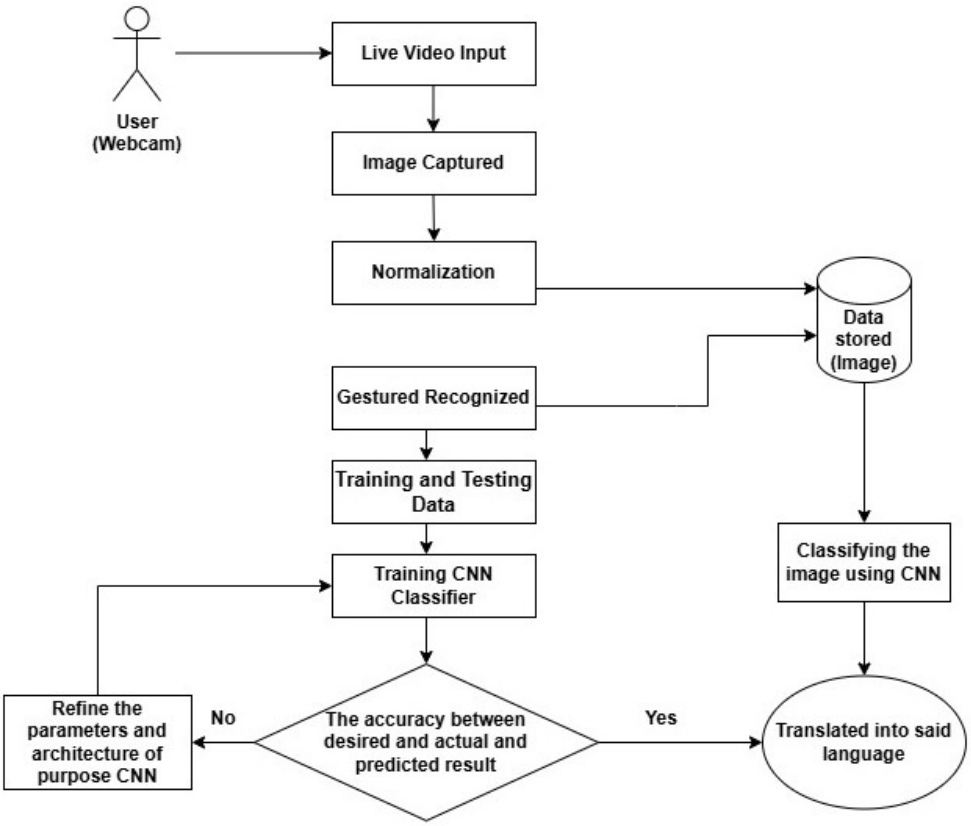


Figure 2: System Architecture Diagram

8.1.2 Use Case Diagram

Use Case is a Diagram that describes the sequence of event of actor using a system to complete the process. The essence is discovering and record- ing functional requirements by writing stories of using a system to help fulfill various Stakeholder goals. Use case is narrative description of domain processes. A use case di- agram shows a set of use cases and actors and their relationships. Use case diagrams are especially important in organizing and modeling the behaviors of a system. Describes what a system does from the standpoint of an external observer.

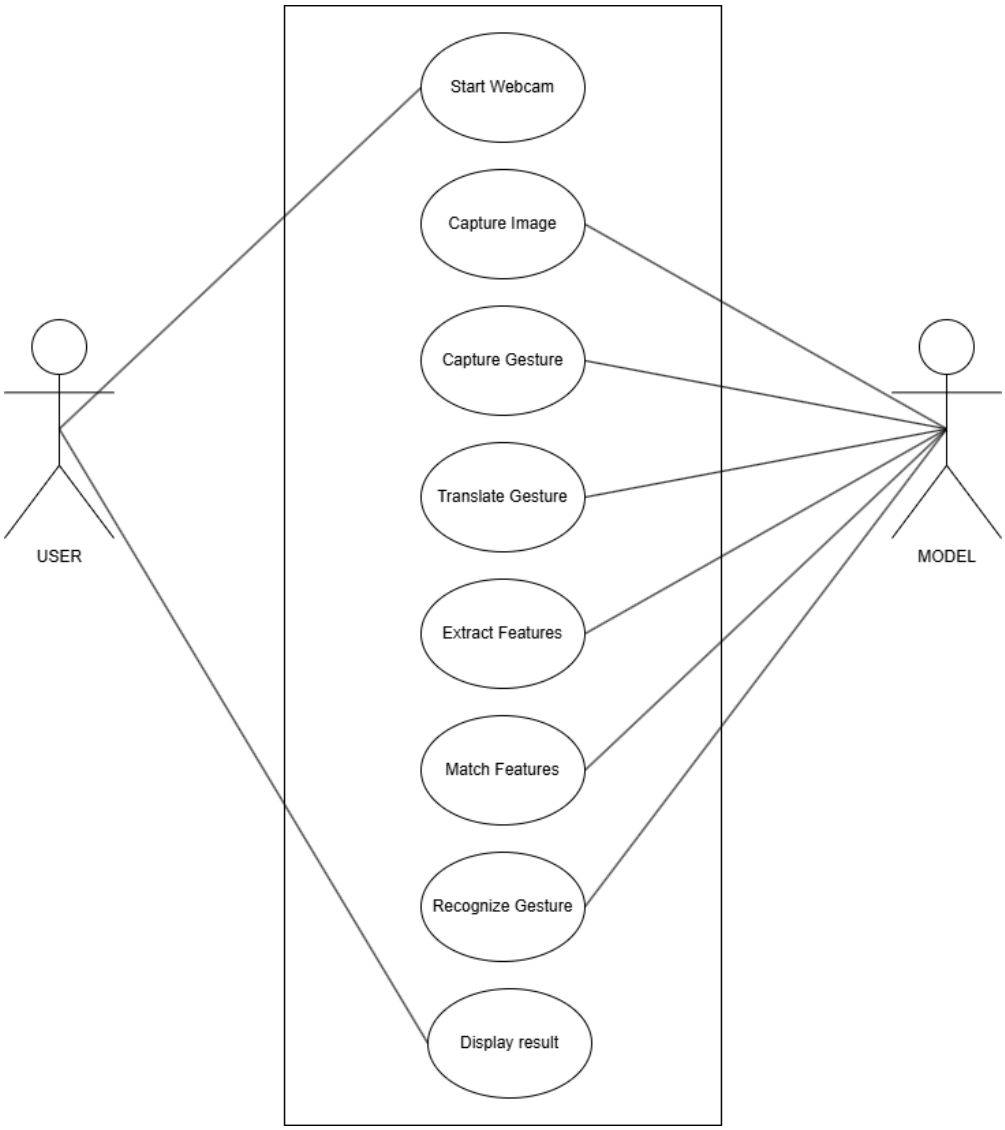


Figure 3: Use Case Diagram

8.1.3 Flow Chart

he kind of diagram that represents the work flow or the process is called flow chart. A flowchart is also defined as a diagrammatic representation of an algorithm, a step-by-step approach to solving a task.

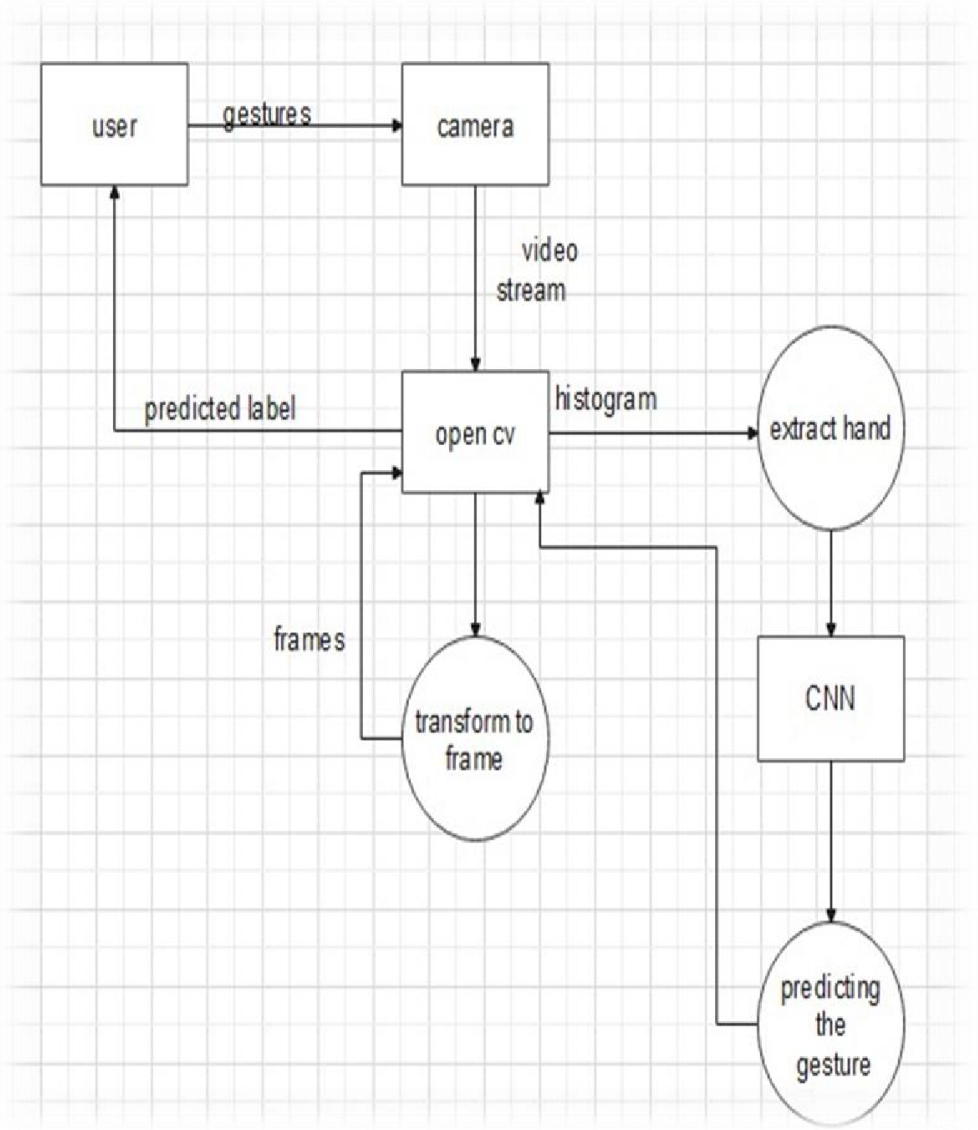


Figure 4: Flow chart

8.1.4 Sequence Diagram

The sequence diagram depicts how several things interact when they are chronologically presented. It refers to the sequence in which messages must be sent and received by view-included objects and other objects required to accomplish class and visual functionality.

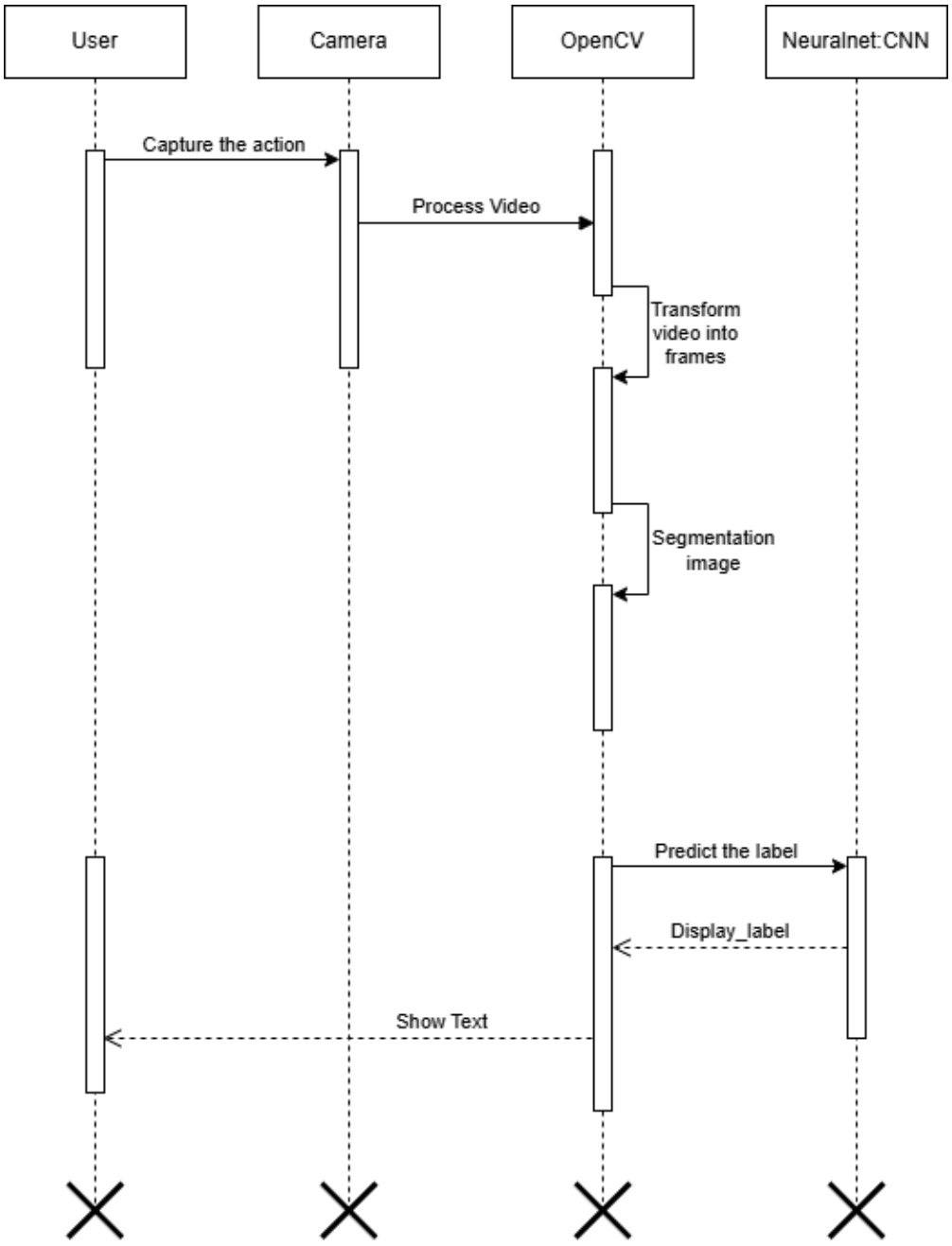


Figure 5: Sequence Diagram

8.1.5 Activity Diagram

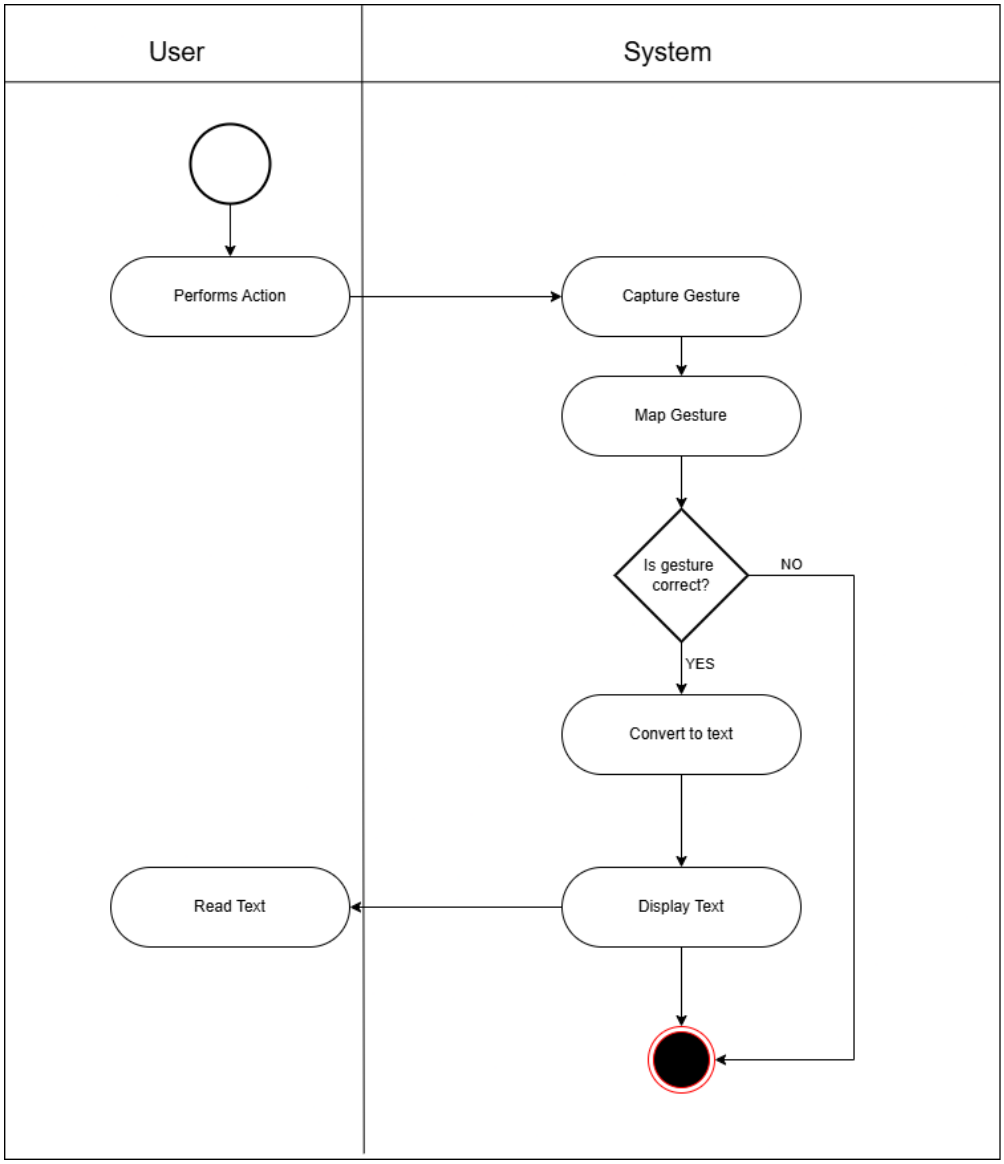


Figure 6: Activity Diagram

9 Task Done

All the tasks done so far regarding this project is listed below.

9.1 Collection Of Data

The initial step was gathering a diverse dataset to train the system. Nearly 1,500 images were collected, encompassing four categories:

A - Hand gesture for the letter A.

B - Hand gesture for the letter B.

C - Hand gesture for the letter C.

Blank - Images representing no gestures, serving as the background class. The images were taken in varying lighting conditions, backgrounds, angels to ensure the model's robustness.



Figure 7: Sample of Dataset

9.2 Splitting The Data

The collected dataset was split into:

Training Set (80%) : Used for learning and optimizing the CNN model. It contains 1135 images.

Validation Set (20%): Used to evaluate the model's performance and tune its parameters. It contains 286 images.

This division ensures the model generalizes well to unseen data and reduces the risk of overfitting.

After split the data looks like this:

Training set for A: 274 images

Training set for B: 278 images

Training set for blank: 288 images

Training set for C: 295 images

Validation set for A: 69 images

Validation set for B: 70 images

Validation set for blank: 73 images

Validation set for C: 74 images

9.3 Creating a web interface using flask

A web interface was developed using Flask to provide a user-friendly interaction platform. Key functionalities include: A live video feed from a webcam for capturing real-time gestures. Integration of the trained model to classify gestures. Displaying the recognized gesture as text in real time. This interface enables seamless interaction between the user and the system.

9.4 Image Preprocessing

Image preprocessing was carried out to prepare the raw dataset for training. Steps involved:

Resizing: All images were resized to a uniform dimension compatible with the CNN input.

Normalization: Pixel values were scaled to a range of 0 to 1 to facilitate faster and more stable training.

Augmentation: Techniques such as rotation, flipping, and brightness adjustments were applied to increase dataset diversity and improve the model's robustness. These preprocessing steps ensured that the data was clean, consistent, and ready for efficient training.

9.5 Creating The CNN model

A Convolutional Neural Network (CNN) was designed to recognize the gestures.

Architecture: Included multiple convolutional layers for feature extraction, pooling layers for dimensionality reduction, and dense layers for classification.

Activation Functions: ReLU for intermediate layers and Softmax for the output layer.

Input Shape: Adjusted to match the preprocessed image dimensions. The model was structured to balance computational efficiency and classification accuracy.

9.6 Training The model

The CNN model was trained on the dataset using the following parameters:

Optimizer: Adam optimizer was used to minimize the loss function efficiently.

Loss Function: Categorical Cross entropy, suitable for multi-class classification.

Metrices: Accuracy was tracked to evaluate model performance. The training process involved iterative fine-tuning to optimize the model's accuracy while ensuring generalizability to new data.

9.7 Combining Everything

In this step, all individual components were integrated into a cohesive system:

Model Deployment: The trained CNN was incorporated into the Flask application for real-time inference.

Live Video Feed Integration: Frames from the webcam were processed and passed to the model for classification.

Dynamic Output Display: The recognized gesture was displayed as text on the web interface in real time.

10 PERFORMANCE ANALYSIS AND VALIDATION

All the graphs showing the accuracy, loss after the completion of epoch is shown below:

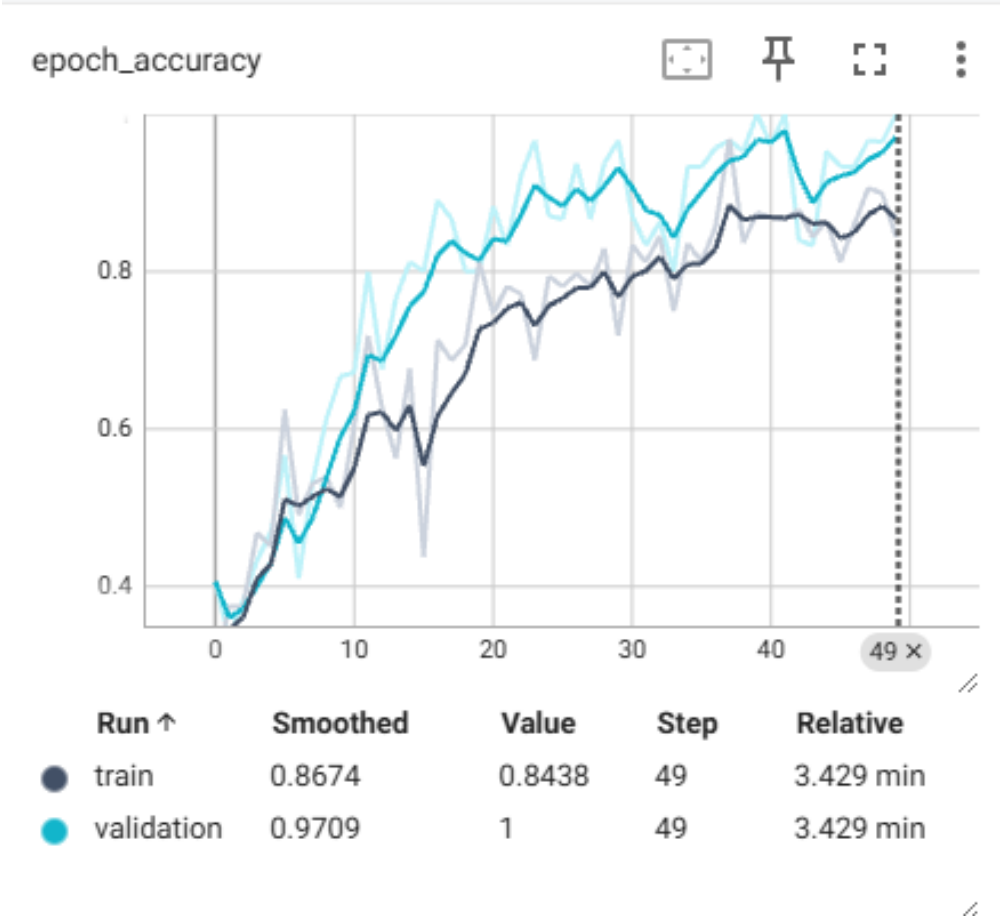


Figure 8: Epoch Accuracy

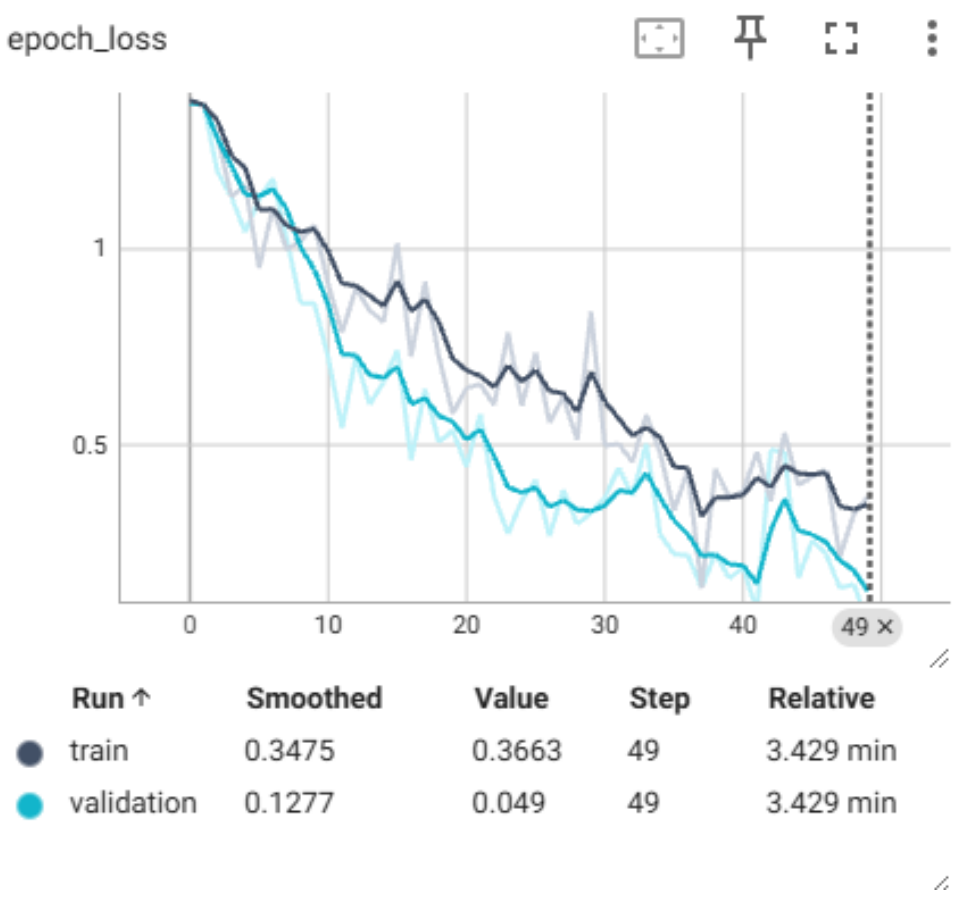


Figure 9: Epoch Loss

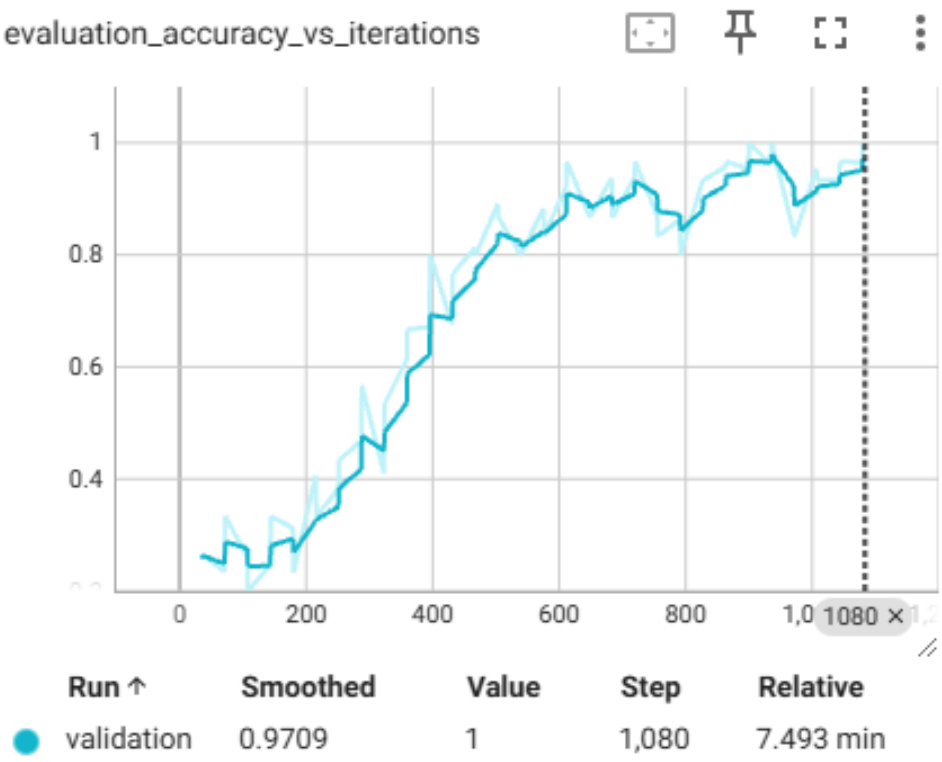


Figure 10: Evaluation Accuracy vs Iterations

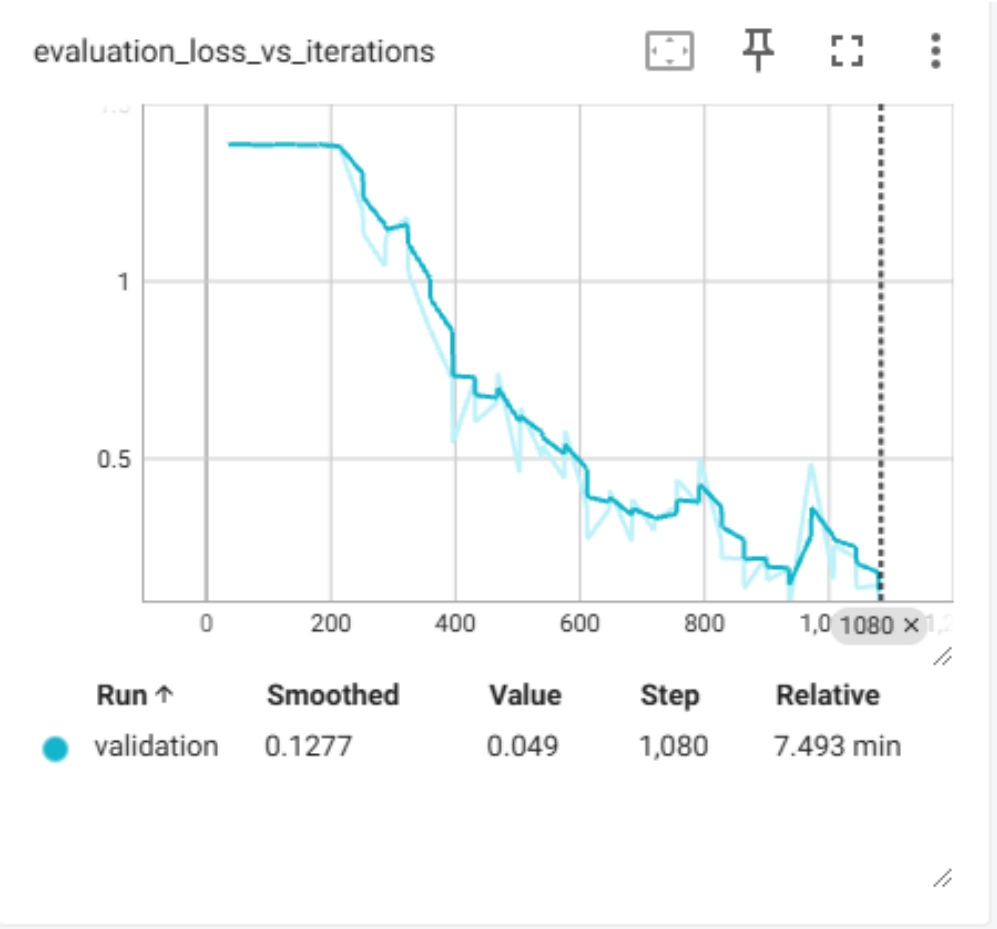


Figure 11: Evaulation loss vs Iterations

10.1 Visualization of our model



Figure 12: CNN model visualization

Input Layer:

- This is where the input images (likely of size 128x128x1 for grayscale images) enter the network.
- The input data is passed to the next layer for feature extraction.

Conv2D Layer:

- Applies a convolution operation using 128 filters, each of size 3x3, to extract spatial features from the input image.
- The kernel operation slides over the input image, detecting patterns like edges, textures, or shapes.

Bias (128):

- Each of the 128 filters has a bias term added to the convolution result.
- Activation: After the convolution, an activation function (likely ReLU) introduces non-linearity.

MaxPooling2D Layer:

- Down-samples the feature maps by taking the maximum value in a 2x2 window, reducing the spatial dimensions.
- This helps retain the most important features while reducing computation and overfitting.

Dropout:

- A dropout layer randomly deactivates a percentage of neurons during training.
- This prevents overfitting by forcing the model to generalize better and not rely on specific neurons.

11 Deliverable

The project delivers a fully functional Sign Language Recognition System capable of real-time recognition of static hand gestures. The system use a webcam to capture input, preprocess the images, and classify gestures using a trained CNN model, providing corresponding text output. Alongside the application, a trained deep learning model optimized for accuracy and generalization is provided, along with any datasets and preprocessing scripts used during development.

12 Conclusion

American Sign Language is one of the most popular sign Languages in the world. In this project we developed a system using a Convolutional Neural Network(CNN) which is capable of recognizing a letters A,B and C of American Sign Language (ASL) gestures successfully.Our system also utilizies the webcam to capture the gestures from the user and translate the input gesture into text format. Our model can predict the letters with approximately 80% accuracy.This system can be carried forward to recognize all the letters of ASL.Also the accuracy of the model can be increased with better datsets and resources support

13 Further works

13.1 Dataset Expansion

Our project can only recognize three letters (A,B,C). We can further expand the dataset to cover the entire ASL alphabet and even commonly used words and signs.

13.2 Improve model accuracy

Due to the limited time and resiurce our model can achieve decent amount of accuracy.We can futher finetune our model with a larger dataset to increase the result accuracy.

13.3 Integration on various field

We can explore the possibilities to integrate this system in many sectors like hospitals,schools,banks and other organizations.

13.4 Real time implementation and feedback

We can conduct the real time testing from actual sign language users and experts to obtain their feedback on usability of the model.Then by acknowledging their feedback we can further improve the system.

14 Project Task and Time Schedule

The time schedule of this project is described below:



Figure 13: Gantt Chart for Iteration 1



Figure 14: Gantt Chart for Iteration 2



Figure 15: Gantt Chart for Iteration 3

References

[1] Shreyasi Dutta and et al. “Sign Language Detection Using Action Recognition in Python”. In: *Neural Computing and Applications* 35.4 (2018), pp. 1234–1250.

[2] R. Harini et al. “Sign Language Translation”. In: *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*. Coimbatore, India: IEEE, 2020, pp. 883–886. DOI: 10.1109/ICACCS48705.2020.9074370.

[3] S. Suresh, H. T. P. Mithun, and M. H. Supriya. “Sign Language Recognition System Using Deep Neural Network”. In: *2019 5th International Conference on Advanced Computing & Communication Systems (ICACCS)*. Coimbatore, India: IEEE, 2019, pp. 614–618. DOI: 10.1109/ICACCI.2019.8728411.

Appendix



Figure 16: Recognition of A

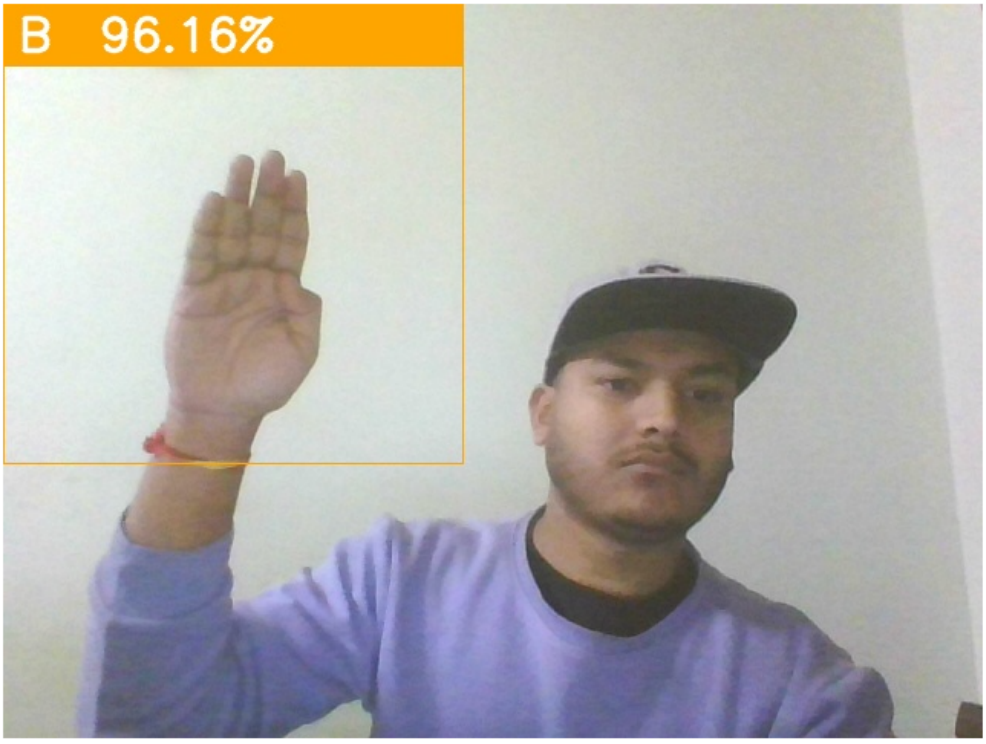


Figure 17: Recognition of B

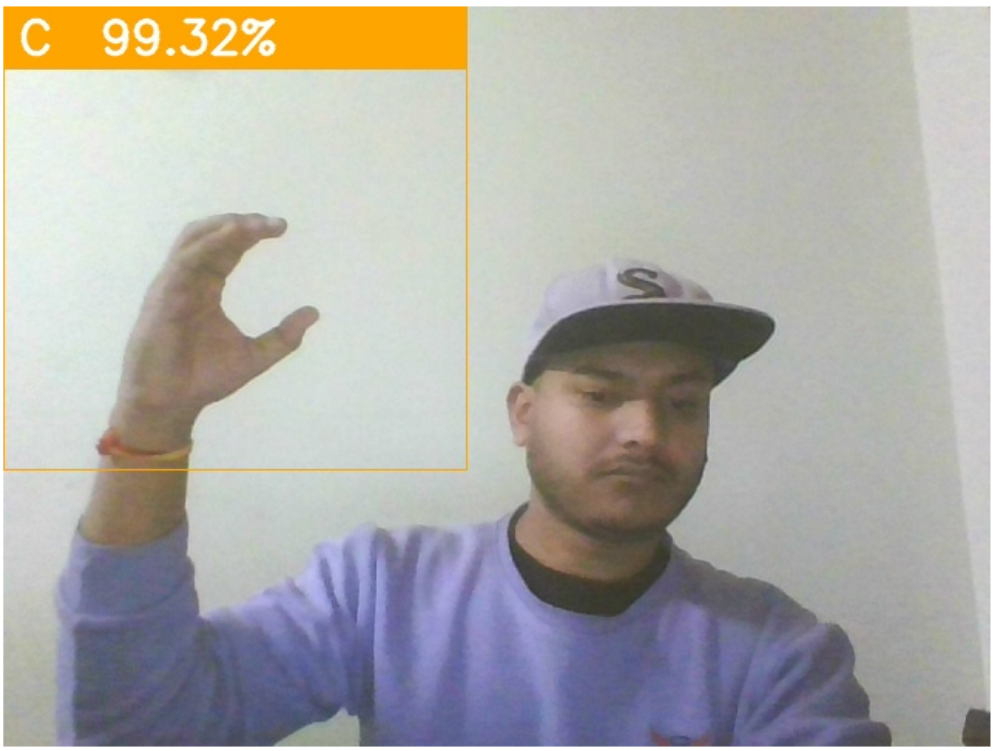


Figure 18: Recognition of C